ESCUELA DE INGENIERÍA
FACULTAD DE INGENIERÍA

EDUCACIÓN PROFESIONAL

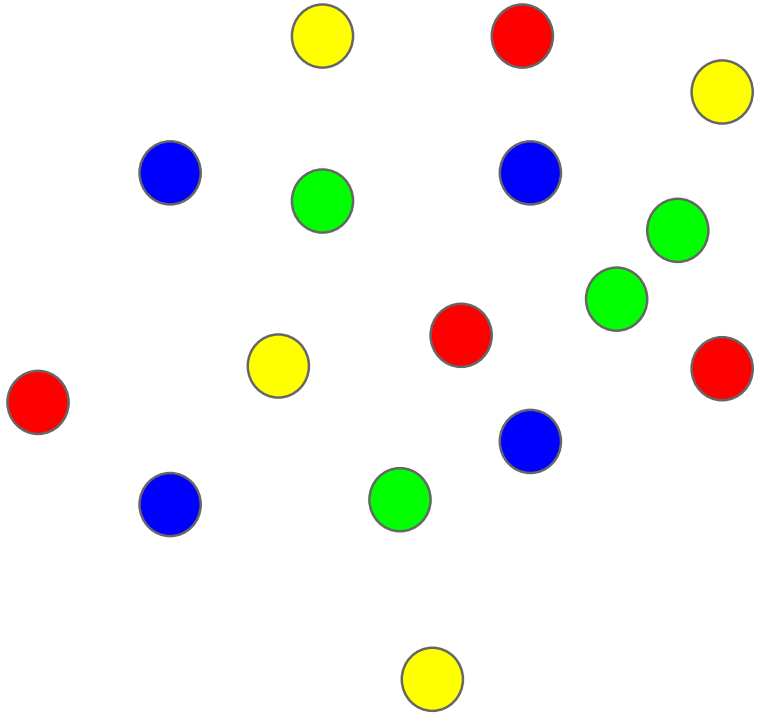
Algoritmos de optimización y learning rate

Diplomado Inteligencia Artificial

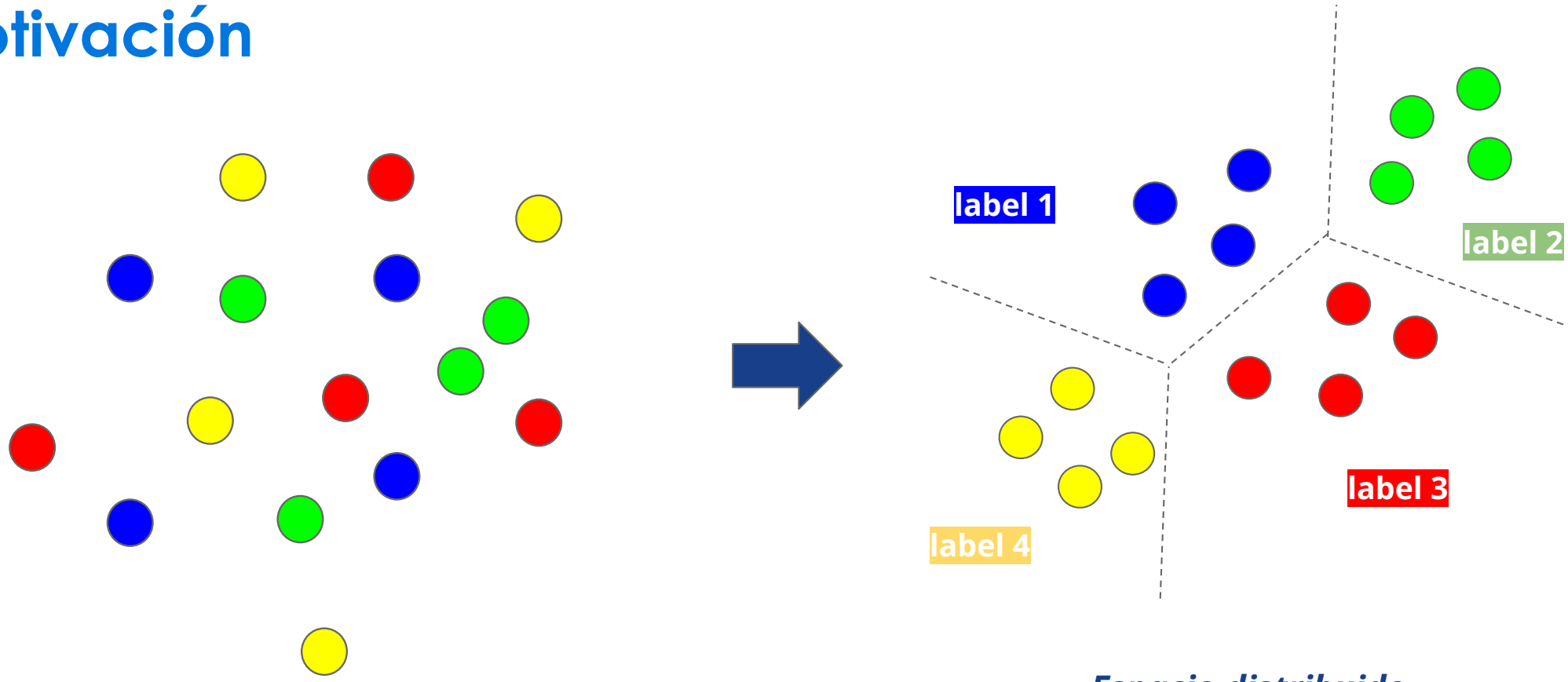
Profesor:
Francisca Cattan Castillo

Motivación

Modelo

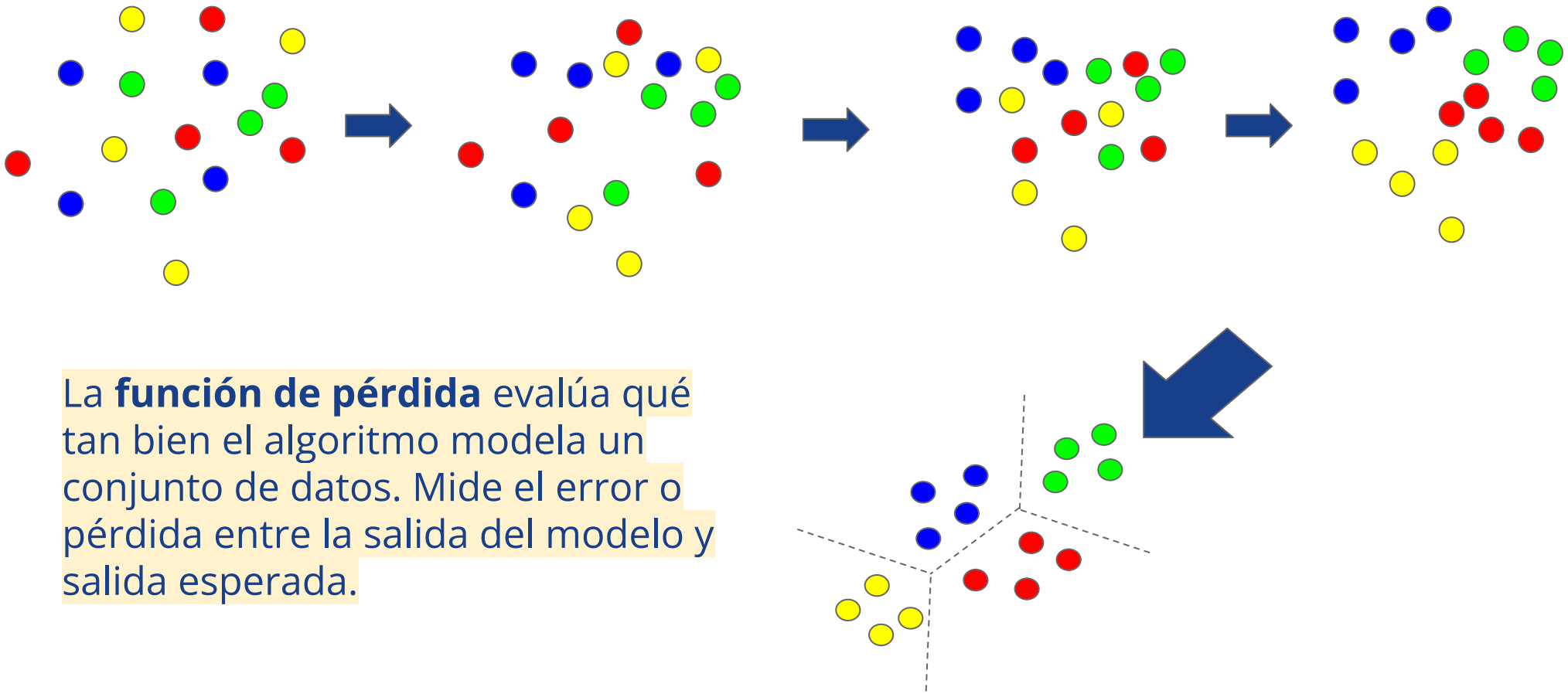


Motivación



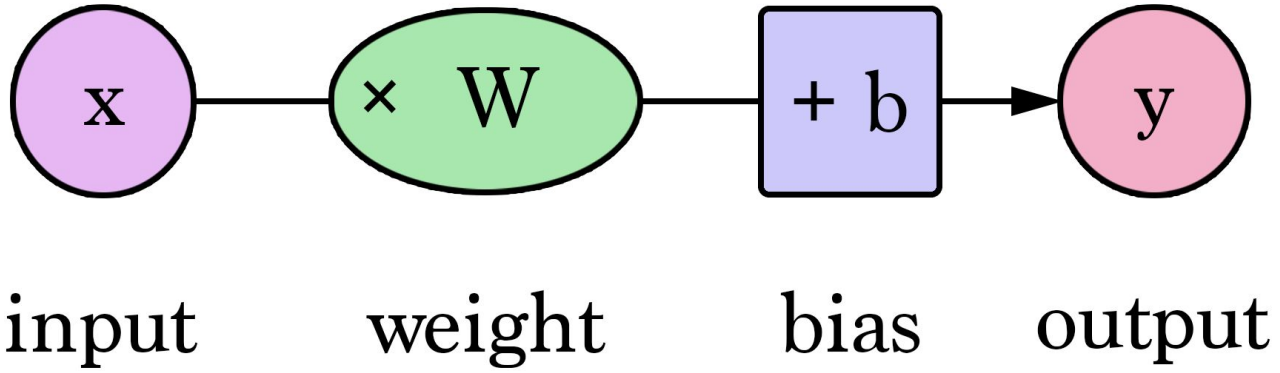
Espacio distribuido

Motivación



La **función de pérdida** evalúa qué tan bien el algoritmo modela un conjunto de datos. Mide el error o pérdida entre la salida del modelo y salida esperada.

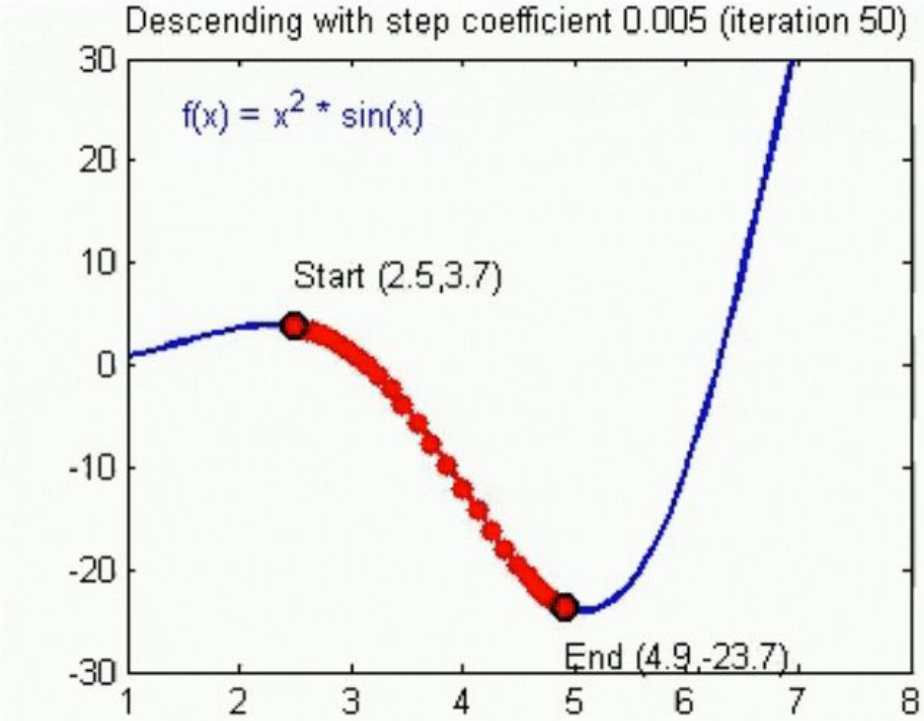
Motivación



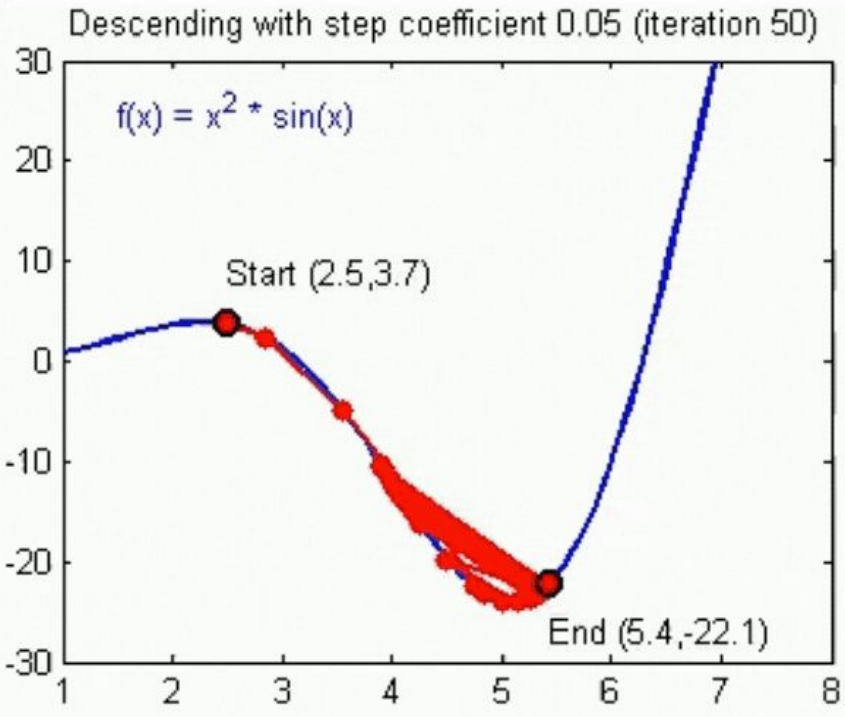
Los ejemplos de entrenamiento pasan por la red para predecir el output. Esto se llama forward pass. La suma de la diferencia entre el valor actual y el predicho es el error. La misión principal de una red neuronal es disminuir el error. **¿Cómo?: Ajustando los pesos de cada nodo.** Esos cambios luego se propagan hacia atrás. Los cambios se calculan con derivadas parciales, pero como los pesos son vectores, les diremos gradientes. Las gradientes nos llevarán a la solución óptima.

Motivación

Convergence



Divergence



Objetivos de Machine Learning

- Tenemos un modelo.

- ¿Cómo adaptamos los pesos del modelo para que se adapten a los datos que tenemos?
  - ¿Cuál es el objetivo del entrenamiento? - tarea
  - ¿Cómo realizamos el entrenamiento?
  - ¿Cómo sabemos en qué dirección debemos mover los pesos?
  - ¿Cómo sabemos cuándo llegamos a una buena solución?
  - ¿Cómo sabemos cuándo parar nuestro entrenamiento?

- No existe una forma directa de obtener los parámetros óptimos para la tarea.

Agenda

01 Concepto de optimización

02 Algoritmo Gradiente Descendente
- ¿Qué es el gradiente?
- ¿Pasos pequeños o pasos más grandes?

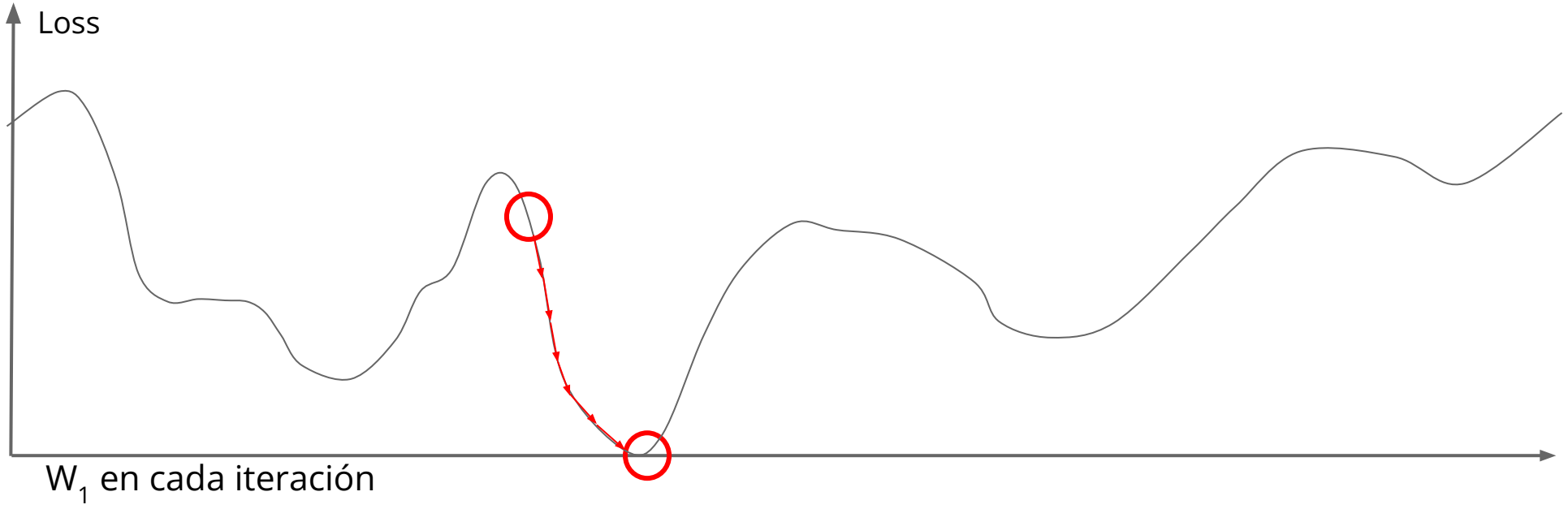
03 Algoritmo SGD: Descenso de Gradiente Estocástico
- Learning Rate
- Momentum
- Nesterov

04 Alternativas a SGD
- AdaGrad
- Adam

05 Adaptar el Learning Rate durante el entrenamiento
- Scheduler
- Early Stopping

Optimización

- El objetivo de un algoritmo de optimización es modificar los pesos del modelo, para **minimizar** la función de pérdida.
- Estos algoritmos son iterativos.
  - Dan pasos que lo acercan a un **óptimo** en cada iteración.



Optimización

valor real

modelo f que recibe x

pesos del modelo

$$L(W) = \underbrace{\mathcal{L}(f(x), y; W)}_{\text{Función de pérdida}} + \underbrace{\alpha\Omega(W)}_{\text{regularizador}}$$

Optimización

- Tenemos una función que queremos optimizar.

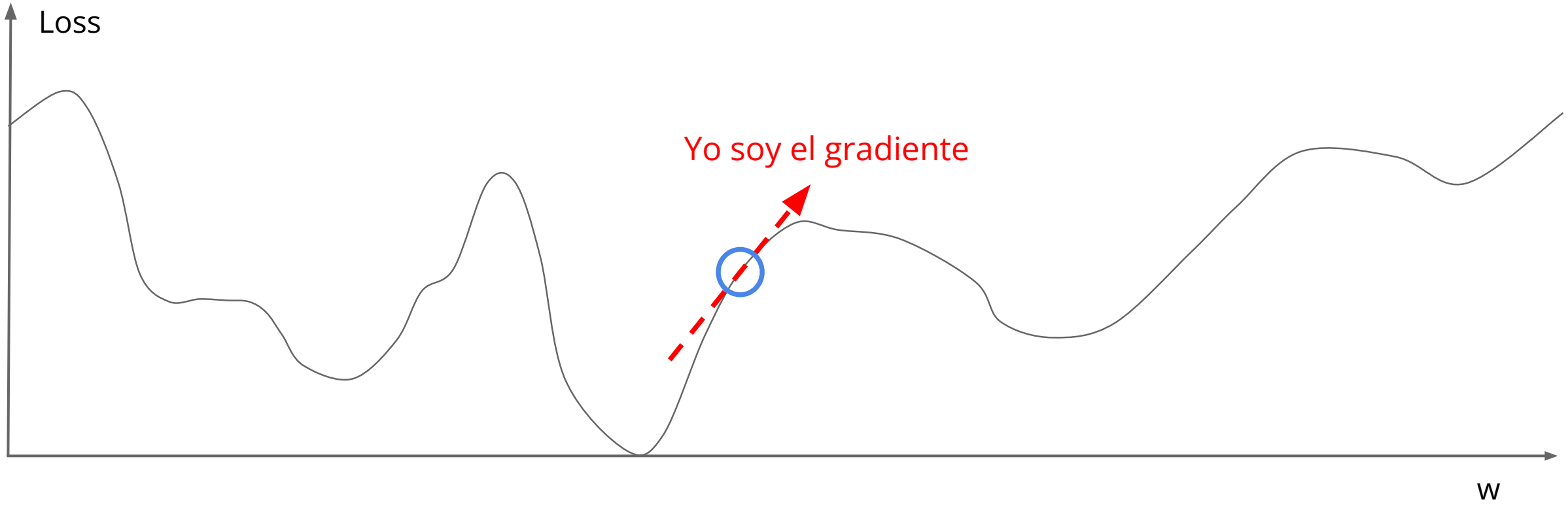
$$L(W) = \mathcal{L}(f(x), y; W) + \alpha\Omega(W)$$

- La **derivada** nos entrega la pendiente de una función.

- El **gradiente** nos indica la dirección en la cual tenemos que mover los pesos para acercarnos a un óptimo.

$$w_i^{new} = w_i^{old} - \eta \boxed{\frac{\partial L}{\partial w_i}}$$ **gradiente**

Optimización

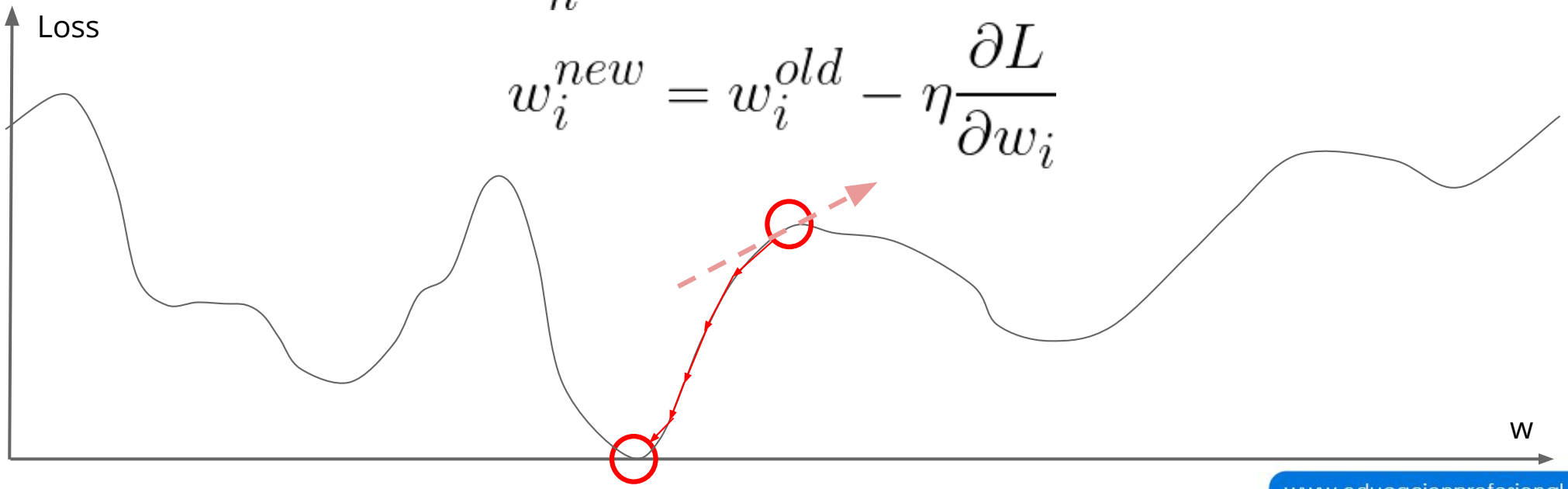


Gradiente Descendente

- Encontramos el valor de la función de pérdida.
  - Considerando todos los elementos de nuestro set de entrenamiento. n son todos los datos.

$$L(W) = \sum_n \mathcal{L}(f(x_n), y_n; W) + \alpha\Omega(W)$$

$$w_i^{new} = w_i^{old} - \eta \frac{\partial L}{\partial w_i}$$



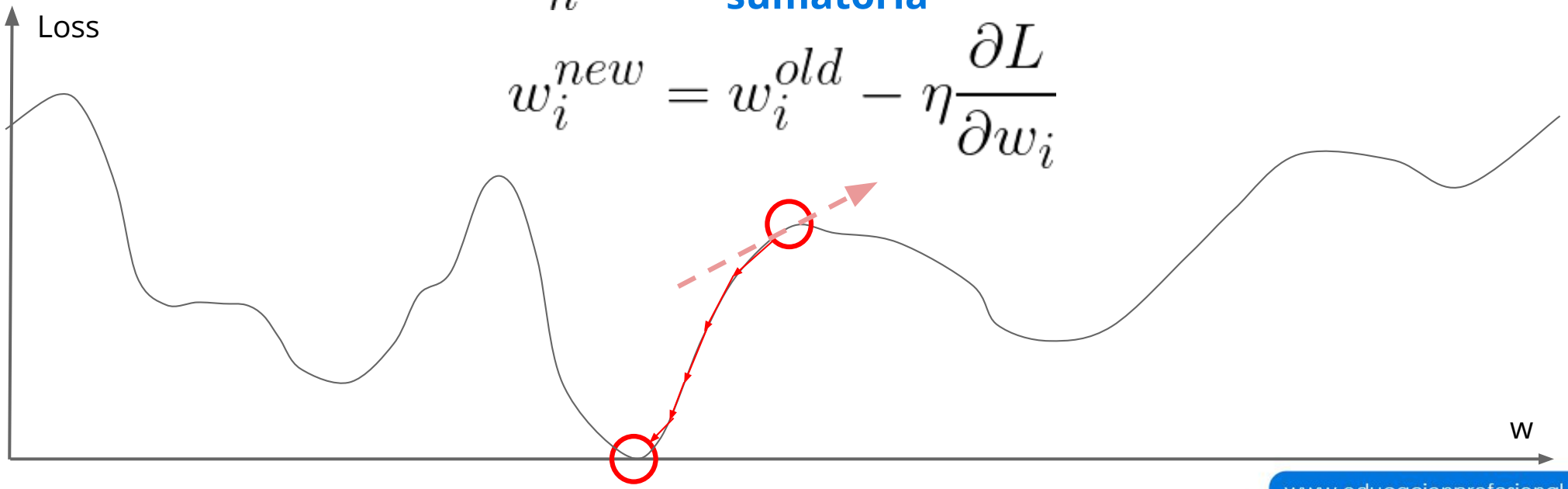
Gradiente Descendente

- Encontramos el valor de la función de pérdida.
  - Considerando todos los elementos de nuestro set de entrenamiento. n son todos los datos.

$$L(W) = \sum_{n} \boxed{\mathcal{L}(f(x_n), y_n; W)} + \alpha\Omega(W)$$

sumatoria

$$w_i^{new} = w_i^{old} - \eta \frac{\partial L}{\partial w_i}$$

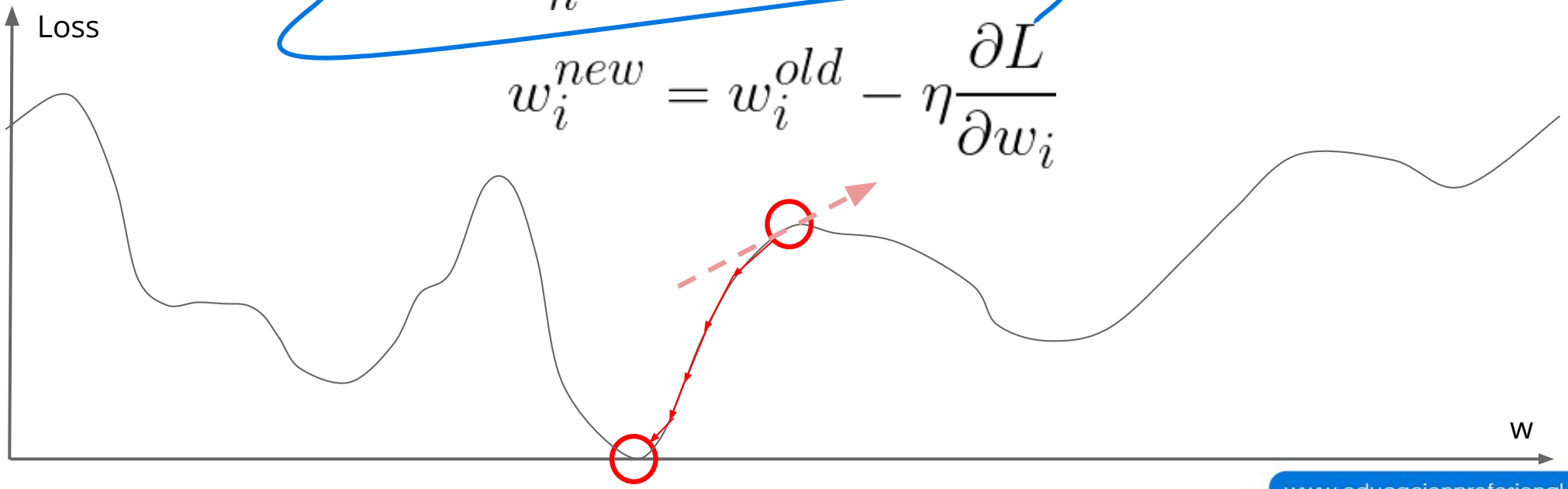


Gradiente Descendente

- Encontramos el valor de la función de pérdida.
  - Considerando todos los elementos de nuestro set de entrenamiento. n son todos los datos.

$$\boxed{L(W)} = \sum_n \mathcal{L}(f(x_n), y_n; W) + \alpha\Omega(W)$$

$$w_i^{new} = w_i^{old} - \eta \frac{\partial L}{\partial w_i}$$



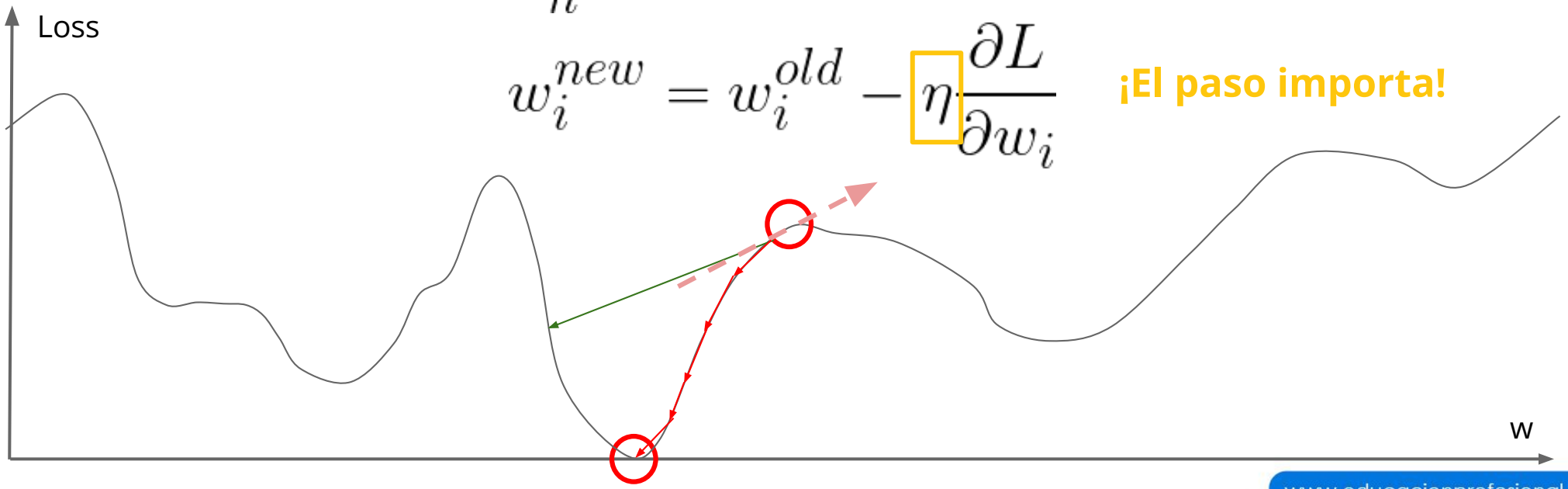
Gradiente Descendente

- Encontramos el valor de la función de pérdida.
  - Considerando todos los elementos de nuestro set de entrenamiento. n son todos los datos.

$$L(W) = \sum_{n} \mathcal{L}(f(x_n), y_n; W) + \alpha\Omega(W)$$

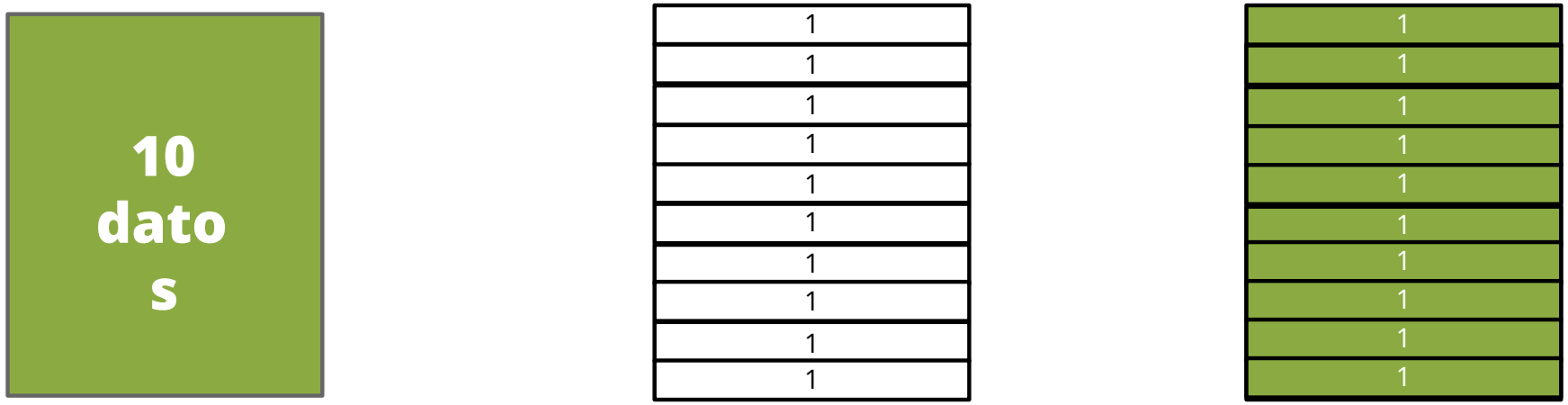
$$w_i^{new} = w_i^{old} - \boxed{\eta}\frac{\partial L}{\partial w_i}$$

¡El paso importa!



SGD: Descenso de Gradiente Estocástico

- Si el dataset que tenemos es muy grande, GD puede ser muy lento.
  - Por cada iteración debemos pasar por todo el dataset.
  - Hacemos muchos cálculos por cada iteración.
- Stochastic Gradient Descent muestrea un subconjuntos de datos
  - A cada **subconjunto** lo llamaremos **batch**.



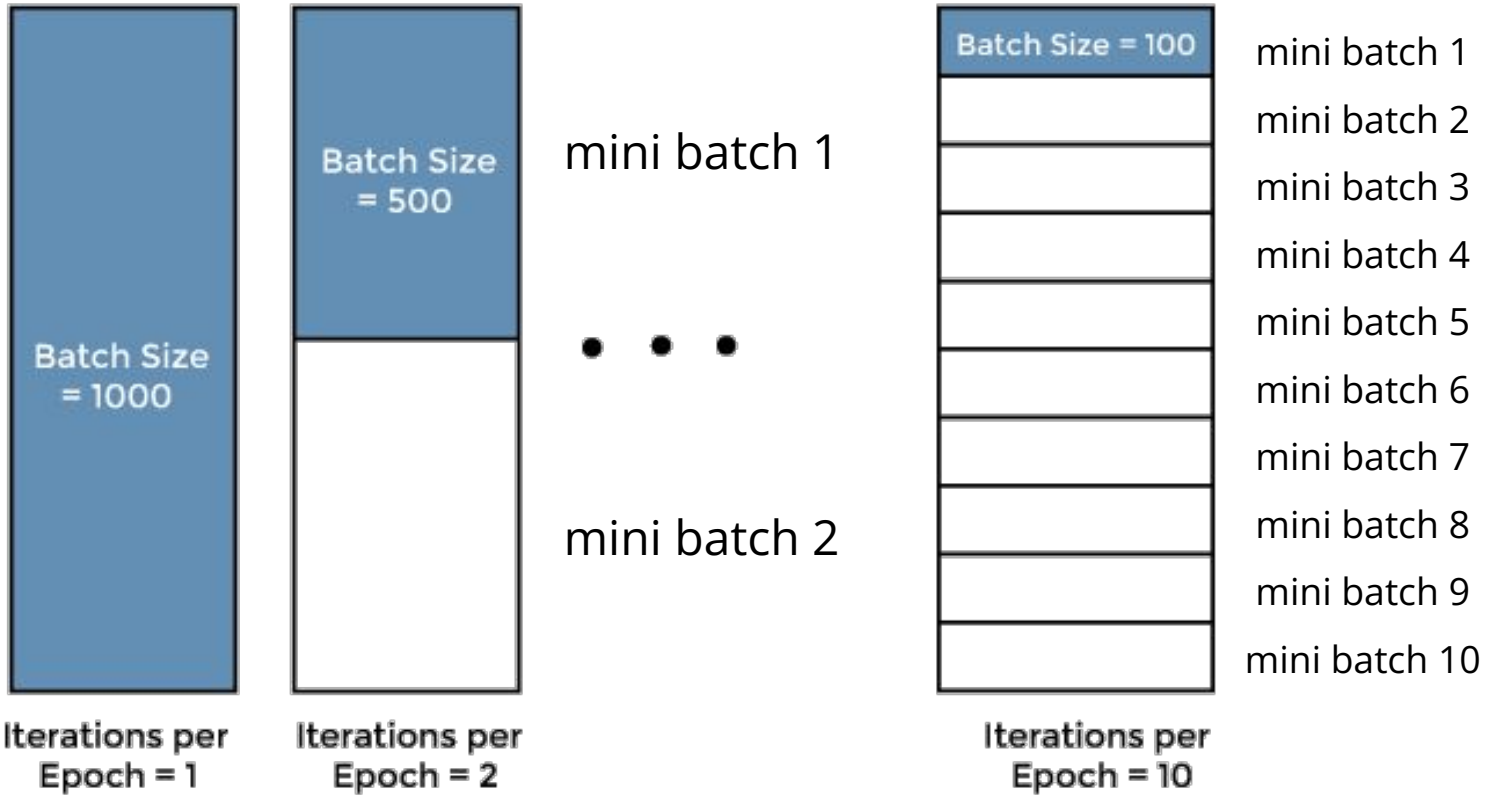
GD: todos en 1 iteración **SGD: todos en 10 iteraciones**

SGD: Descenso de Gradiente Estocástico

Una **época** es un ciclo en donde el modelo pasa por todo el set de datos.

Aquí vemos la relación iteración y época.

En teoría, más épocas indica mayor diversidad de patrones y mejor generalización.



SGD: Descenso de Gradiente Estocástico

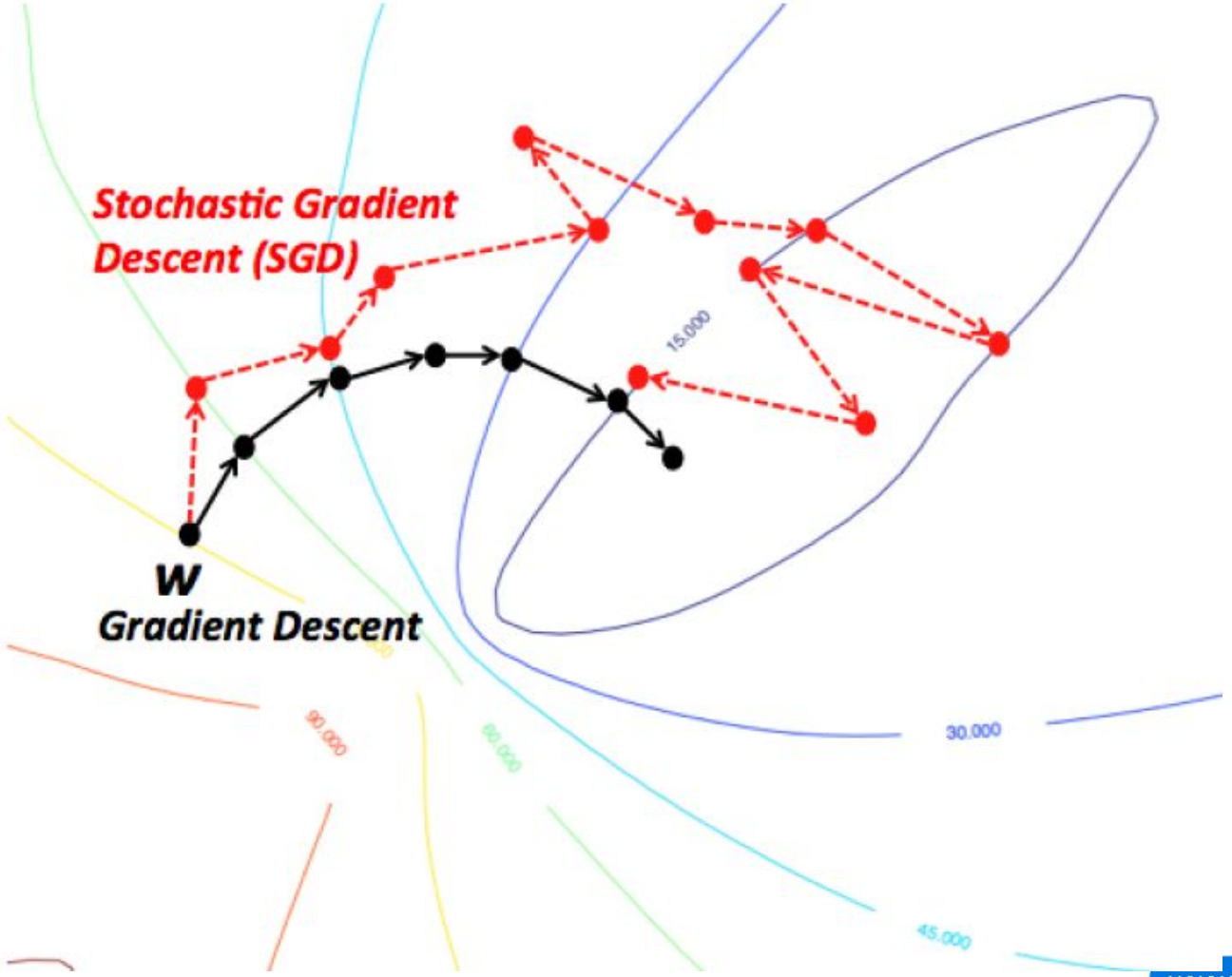
SGD nos permite acelerar el proceso de convergencia.

- n ahora es 1 dato, o un **minibatch**.
- Actualizamos más veces en menos tiempo.

$$L(W) = \sum_{n} \mathcal{L}(f(x_n), y_n; W) + \alpha\Omega(W)$$

$$w_i^{new} = w_i^{old} - \eta \frac{\partial L}{\partial w_i}$$

SGD: Descenso de Gradiente Estocástico



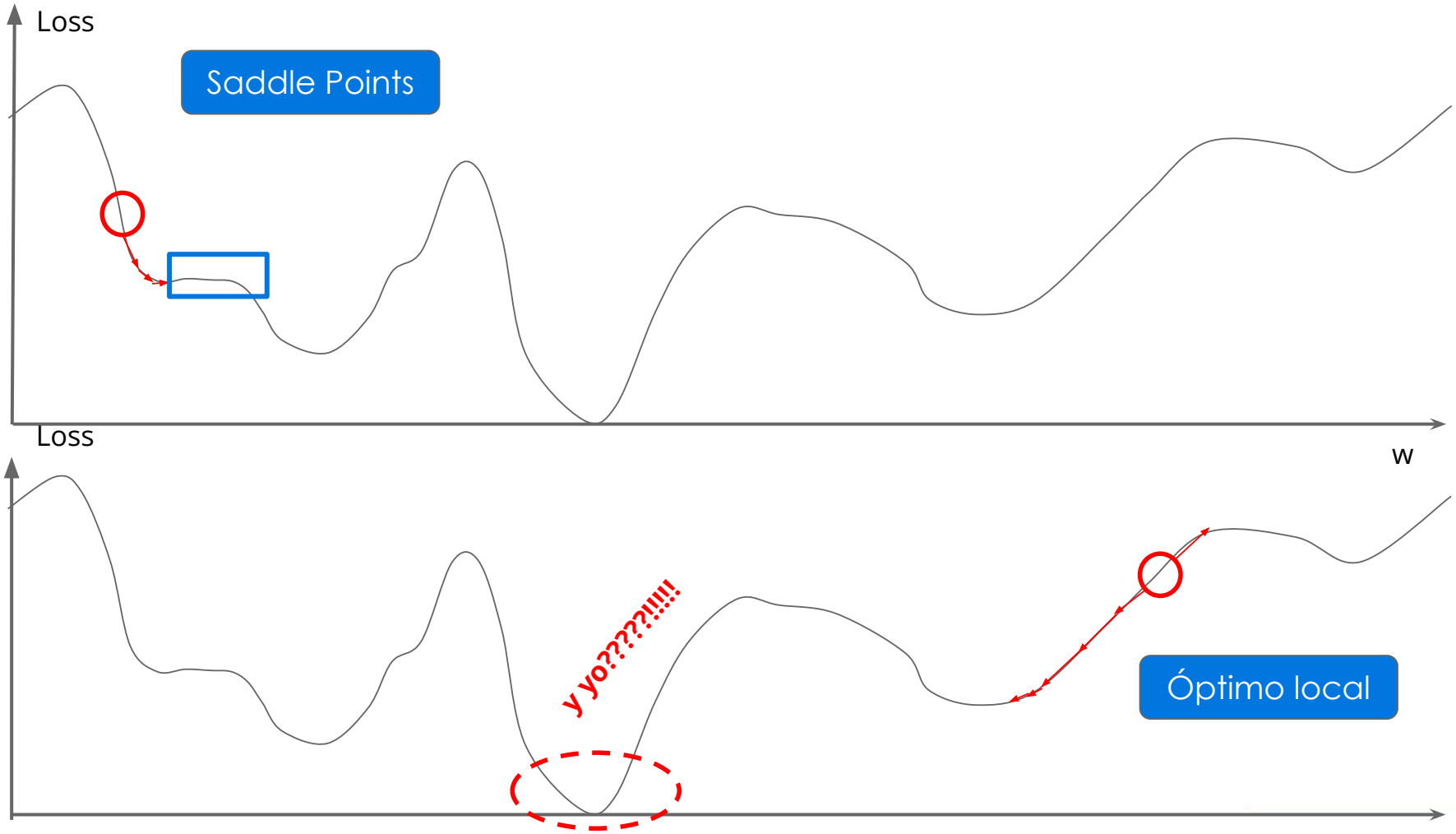
SGD: Problemas

Saddle Points

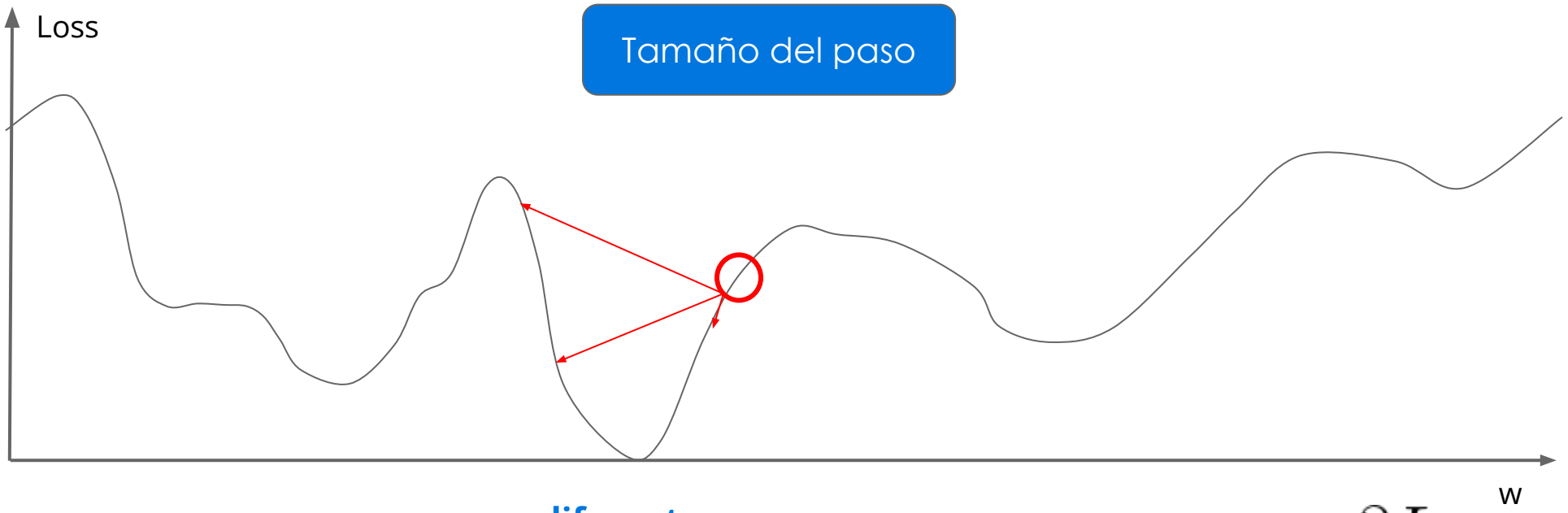
Óptimos locales

Tamaño de los pasos

SGD: Problemas



SGD: Problemas

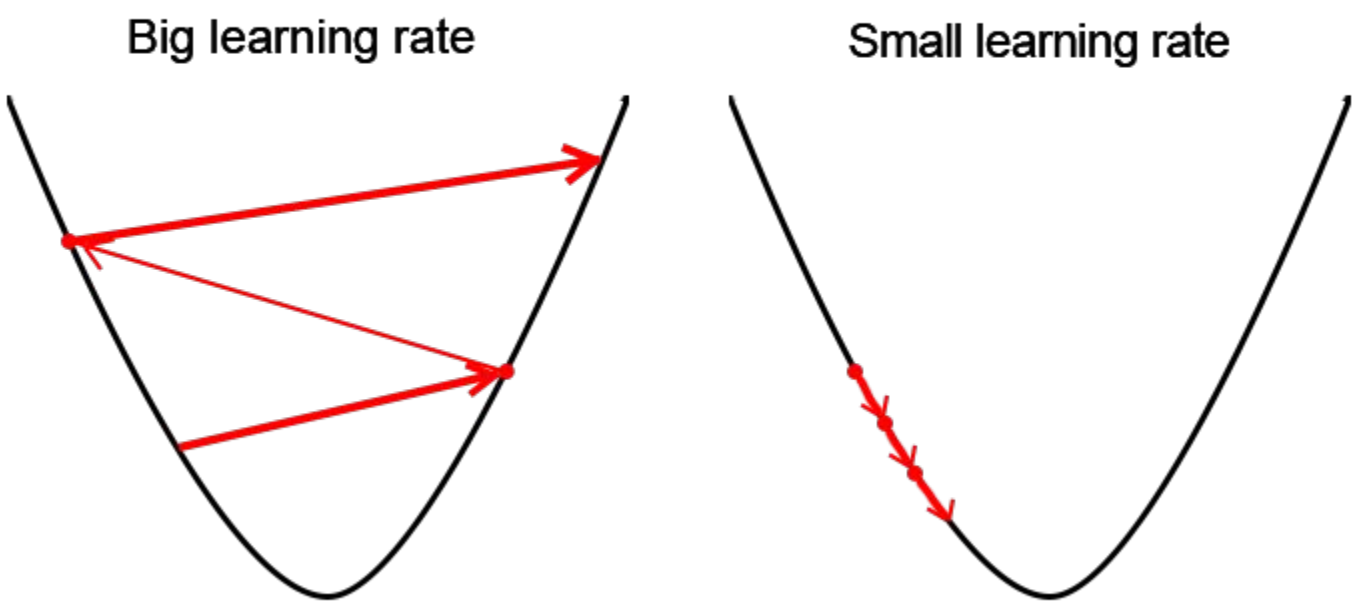


diferentes tamaños de pasos

$$w_i^{new} = w_i^{old} - \boxed{\eta} \frac{\partial L}{\partial w_i}$$

SGD: Problemas

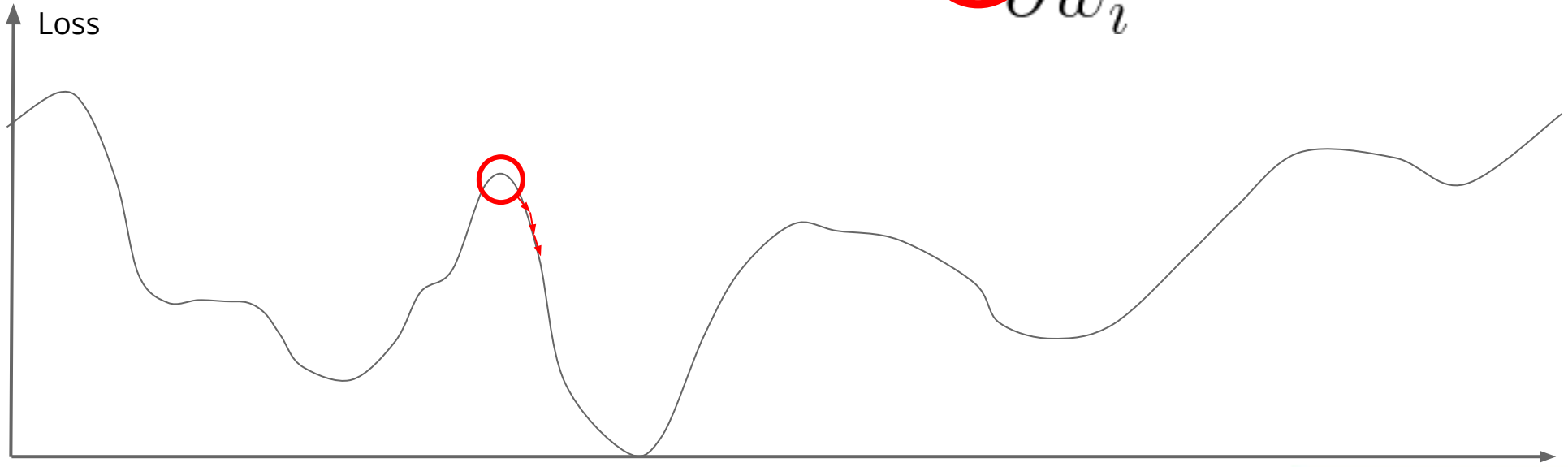
Tamaño del paso



SGD: Learning Rate

- ¿Qué tan grande debe ser el paso que damos?
  - ¿Pasos grandes o pasos pequeños?

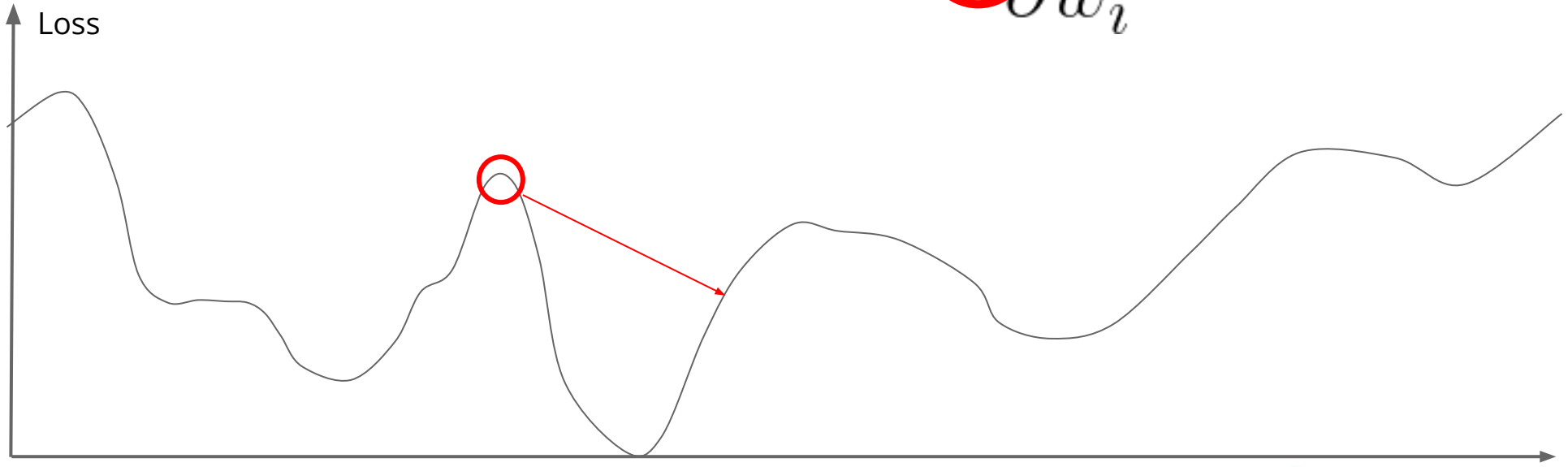
$$w_i^{new} = w_i^{old} - \eta \frac{\partial L}{\partial w_i}$$



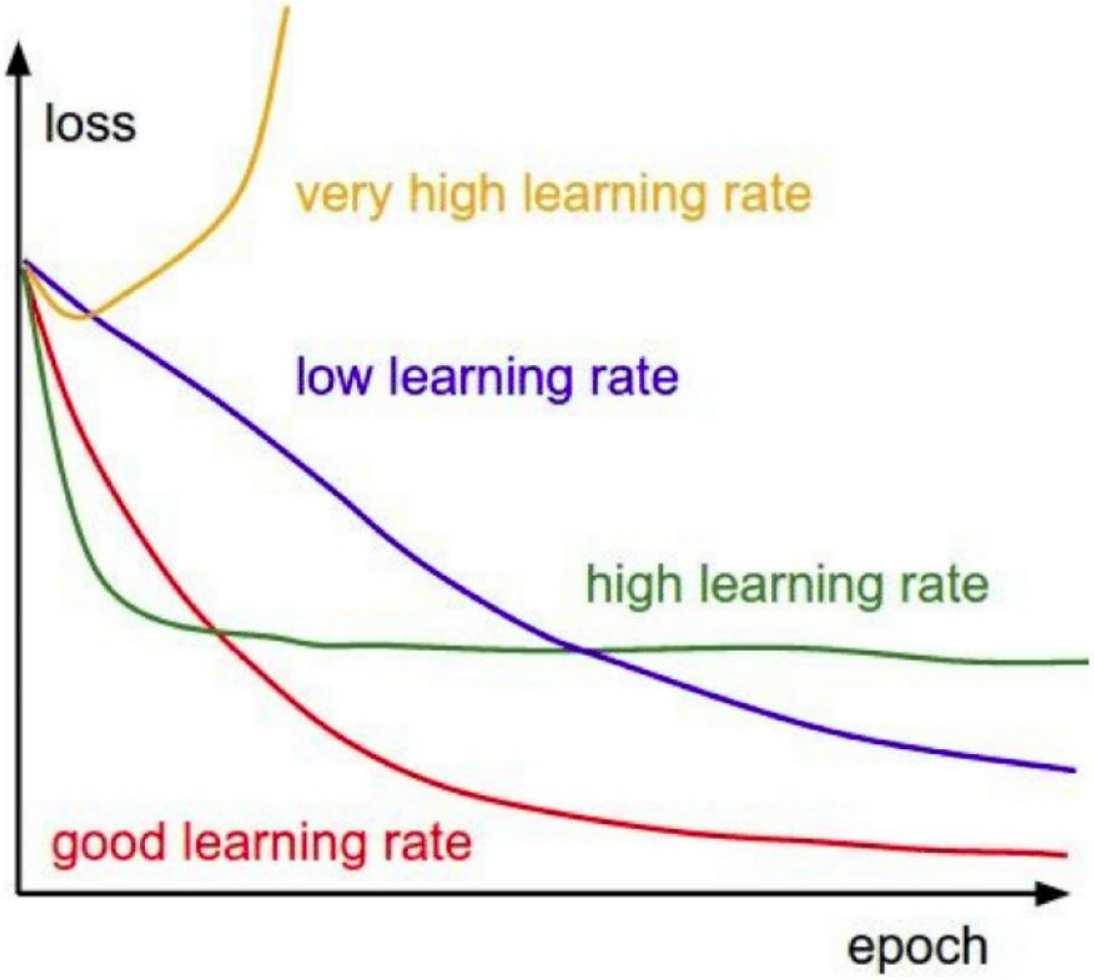
SGD: Learning Rate

- ¿Qué tan grande debe ser el paso que damos?
  - ¿Pasos grandes o pasos pequeños?

$$w_i^{new} = w_i^{old} - \eta \frac{\partial L}{\partial w_i}$$



SGD: Learning Rate



SGD: Momentum

- Momentum agrega la idea de mantener la dirección del gradiente.
  - Agregamos historia de cómo se ha movido el gradiente en el pasado.

$$w_{t+1} = w_t - \eta \nabla f(x_t)$$ **update sin momentum**

$$w_{t+1} = w_t - \eta \boxed{v_{t+1}}$$ **update con momentum**

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

- Suavizamos el valor del gradiente con los valores pasados.
  - Similar al momentum físico.
  - Se acumula velocidad en dirección consistente.
  - Valor de ρ va de 0.0 a 1.0

SGD: Momentum

Ejemplo de una sola dimensión.

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$
$$w_{t+1} = w_t - \eta v_{t+1}$$

$\rho = 0.1$

Batch	Gradiente	v_{t+1}
Batch 1	-2	-2
Batch 2	-4	-4.2
Batch 3	-3	-3.42
Batch 4	2	1.658

SGD: Momentum

Ejemplo de una sola dimensión.

0.1*-2 + -4

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$w_{t+1} = w_t - \eta v_{t+1}$$

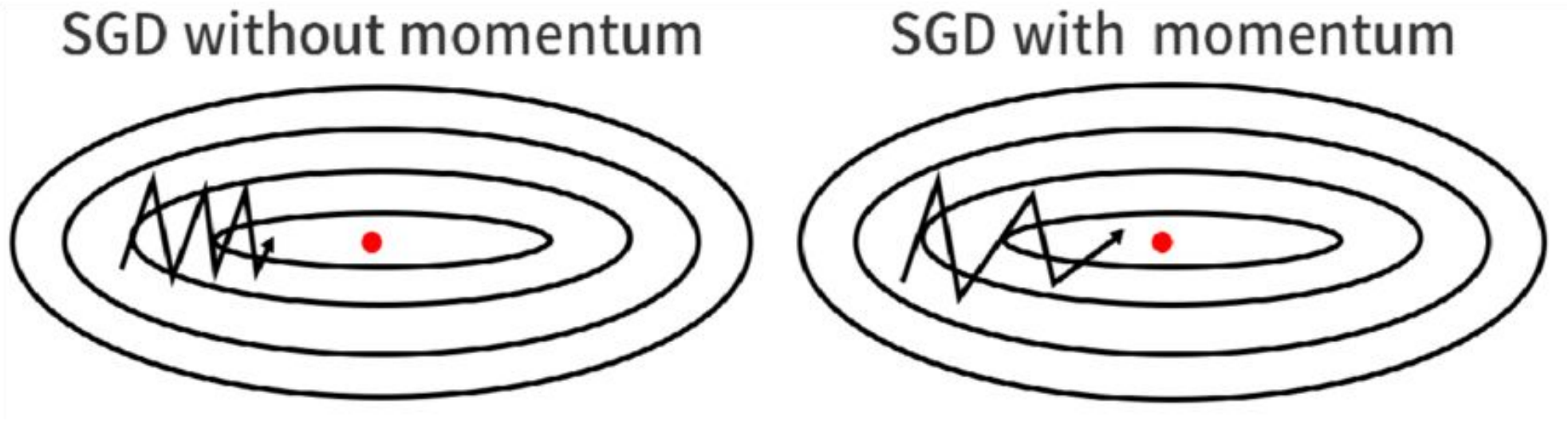
$\rho = 0.1$

Batch	Gradiente	v_{t+1}
Batch 1	-2	-2
Batch 2	-4	-4.2
Batch 3	-3	-3.42
Batch 4	2	1.658

SGD: Momentum

Una solución al problema de que los gradientes dependen de solo un batch.

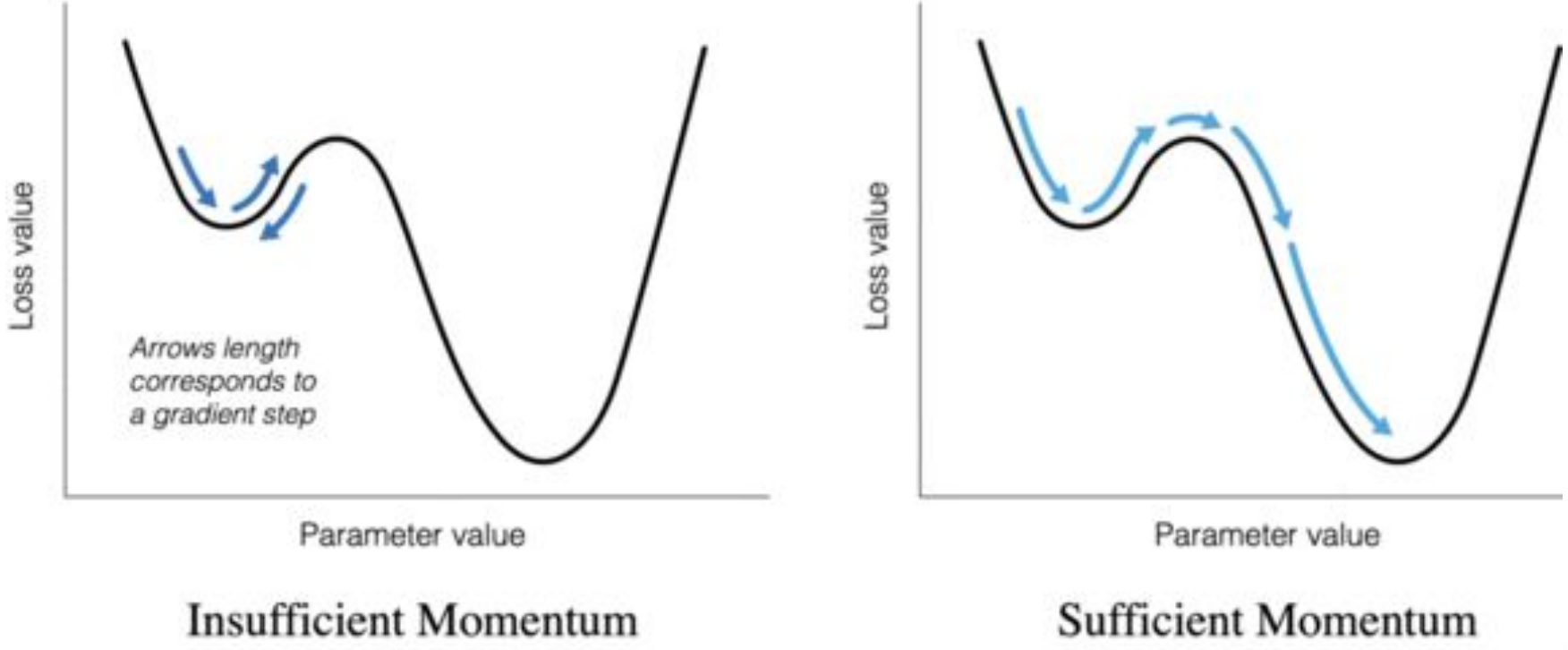
- Distintos grupos de datos puede tener diferentes gradientes.



SGD: Momentum

Una solución al problema de que los gradientes dependen de solo un batch.

- Distintos grupos de datos puede tener diferentes gradientes.



SGD: Nesterov

Una forma alternativa de ver el momentum es utilizando Nesterov Accelerated Gradient (NAG).

- Buscamos hacer la corrección del momentum antes de encontrar el gradiente.
- Seguimos la dirección dado el momentum que tenemos.

$$w_t = w_t - \rho v_t$$ **update momentum actual**

$$v_{t+1} = \rho v_t - \alpha \frac{\partial L(x)}{\partial w_t}$$ **corrección del gradiente**

$$w_{t+1} = w_t + v_{t+1}$$ **update final, se hace el salto al nuevo peso**

SGD: Nesterov

Es una visión a futuro, donde la velocidad (momentum) nos va a llevar.

$$\rho v_t$$

Agregamos una tasa de corrección, que es el gradiente.

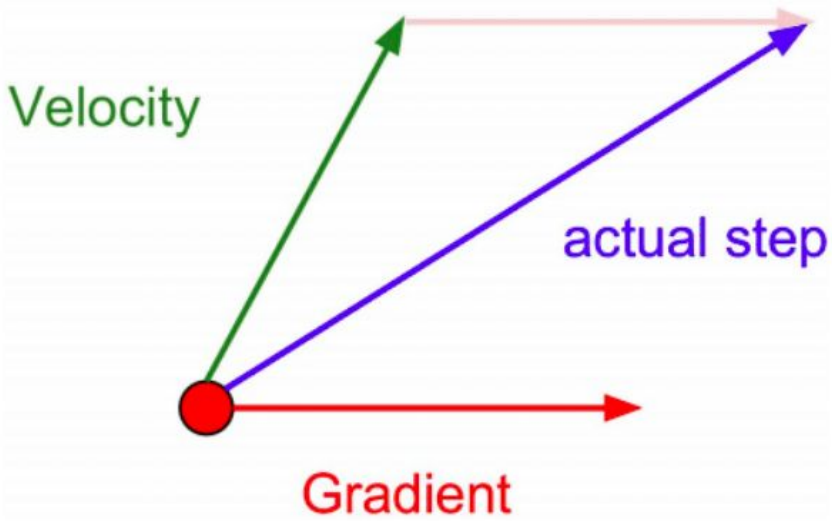
$$\alpha \frac{\partial L(x)}{\partial w_t}$$

Luego nos movemos con la suma.

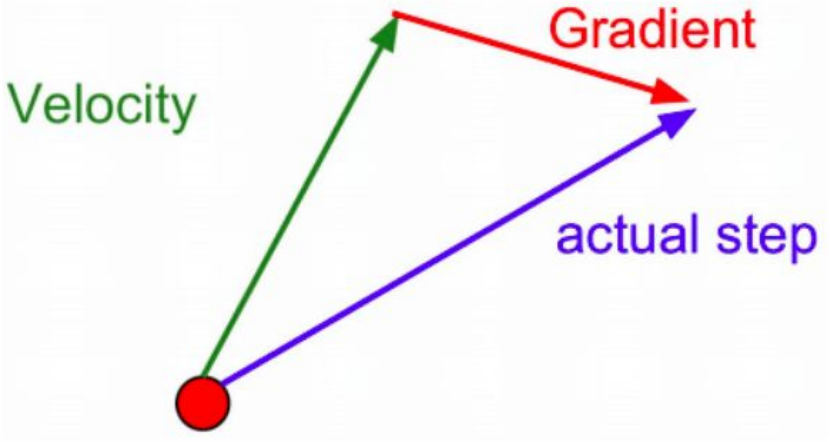
$$w_{t+1} = w_t + v_{t+1}$$

SGD: Momentum y Nesterov

Momentum



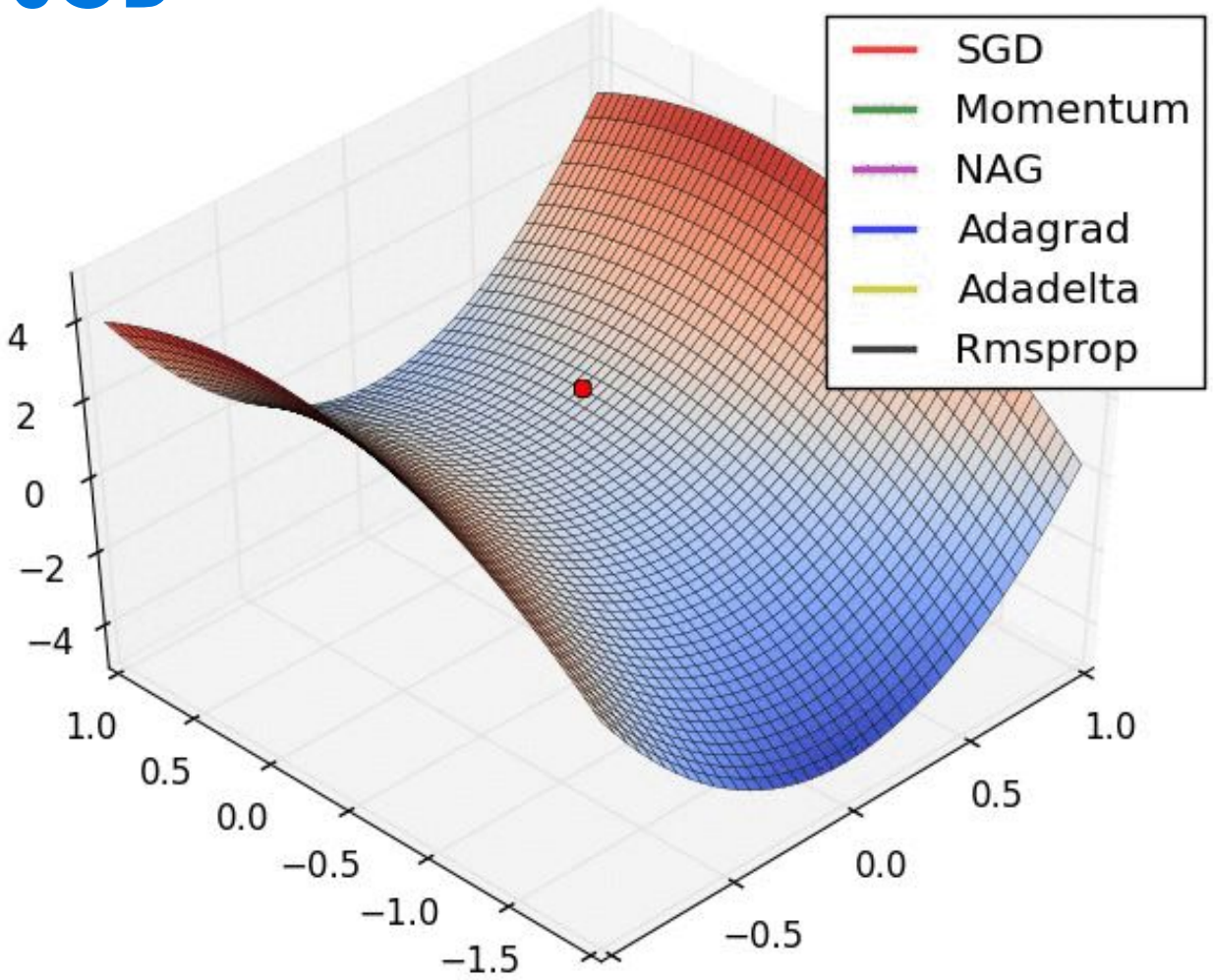
Nesterov



SGD: Momentum y Nesterov

Característica	Momentum Clásico	Nesterov (NAG)
Punto de cálculo	Se calcula en la posición actual.	Se calcula en la posición **"predecida"**.
Comportamiento	Es como una bola bajando un cerro que no sabe qué hay más abajo.	Es como un esquiador que mira hacia adelante para frenar antes de una curva cerrada.
Convergencia	Rápida, pero puede oscilar al final.	Más estable porque evita que el modelo "sobrepase" el mínimo tan agresivamente.

Alternativas a SGD



Adaptive Gradient (AdaGrad)

- Cada peso tiene su propio learning rate, que se comporta diferente en el tiempo.
  - Dependiendo del batch.
- La idea de AdaGrad es ir actualizando el learning rate de cada peso del modelo según su historia.

$$w_t^i = w_{t-1}^i - \eta_{w^i} \frac{\partial L(x)}{\partial w^i}$$

$$\eta_{w^i} = \frac{\eta}{\sqrt{\sum_i^t (G_i)^2}}$$

Adaptive Gradient (AdaGrad)

- Cada peso tiene su propio learning rate, que se comporta diferente en el tiempo.
  - Dependiendo del batch.
- La idea de AdaGrad es ir actualizando el learning rate de cada peso del modelo según su historia.

$$w_t^i = w_{t-1}^i - \eta_{\boxed{w^i}} \frac{\partial L(x)}{\partial w^i}$$

$$\eta_{w^i} = \frac{\eta}{\sqrt{\sum_i^t (G_i)^2}}$$

Adaptive Gradient (AdaGrad)

- Es una especie de normalización de los gradientes.
  - Gradientes muy pronunciados son reducidos.
  - Gradientes muy bajos son acelerados.
- ¿Qué pasa con el valor del learning rate en el tiempo?
  - Tiende a cero.
  - El problema es que tiende a reducir muy rápidamente el learning rate.

Adaptive moments (Adam)

- Es una combinación entre SGD con momentum y AdaGrad (escalamiento).

$$w_i^t = w_i^{t-1} - \eta \frac{\boxed{r_t}}{\boxed{\sqrt{v_t} + \epsilon}}$$

Momentum

Escalamiento del momentum

- Donde el primer momento es la **media** y el segundo la **varianza**.

$$r_t = (1 - \gamma_1) G_i^t + \gamma_1 r_{t-1}$$

$$v_t = (1 - \gamma_2)(G_i^t)^2 + \gamma_2 v_{t-1}$$

Adaptive moments (Adam)

- Se realiza una corrección a ambos momentos, para arreglar que ambos estimadores parten de cero.

$$r_t = \frac{(1-\gamma_1)G_i^t + \gamma_1 r_{t-1}}{1-\gamma_1^t}$$

$$v_t = \frac{(1-\gamma_2)(G_i^t)^2 + \gamma_2 v_{t-1}}{1-\gamma_2^t}$$

$$w_i^t = w_i^{t-1} - \eta \frac{r_t}{\sqrt{v_t} + \epsilon}$$

Adaptive moments (Adam)

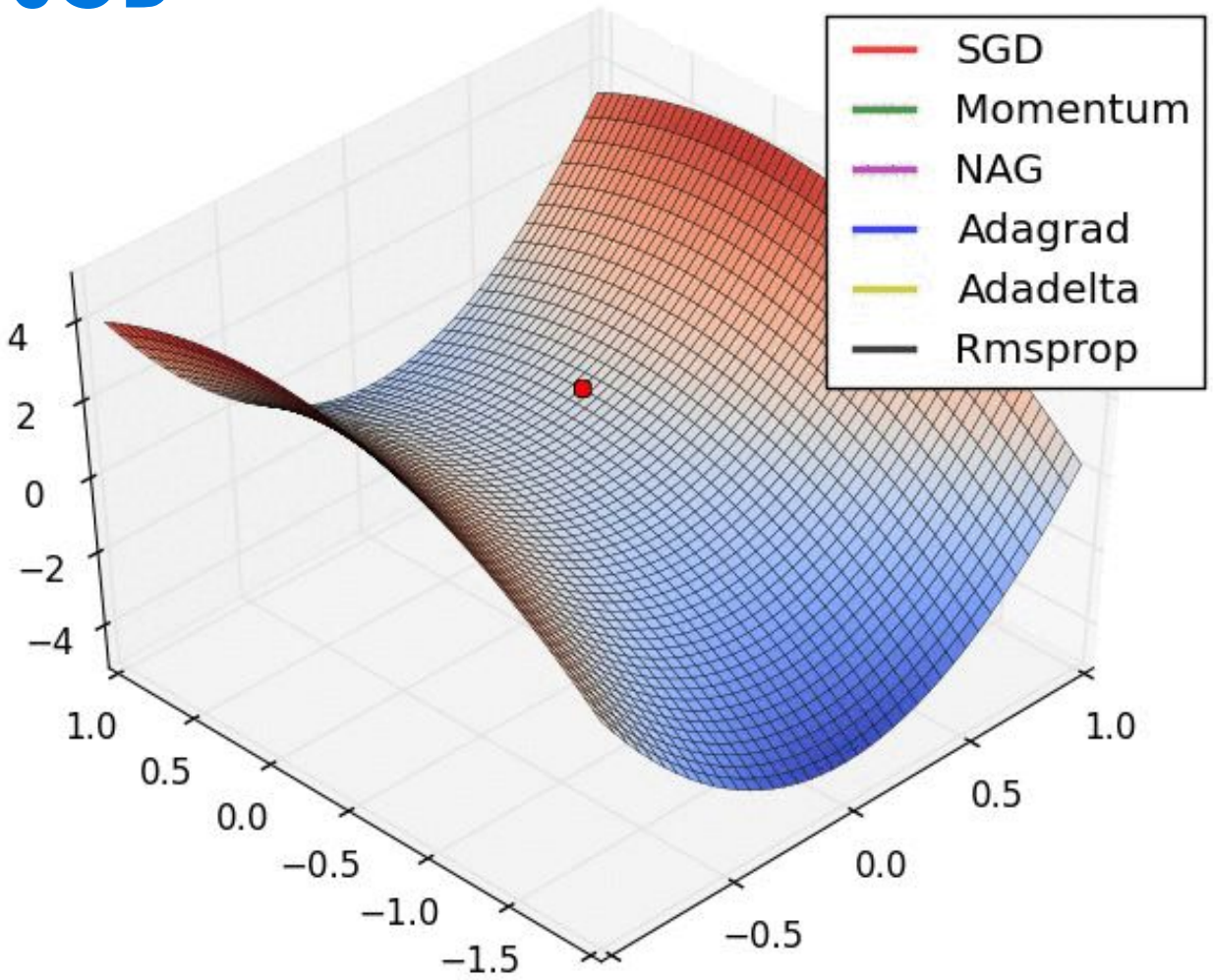
- Es una combinación entre SGD con momentum y AdaGrad (escalamiento).

$$w_i^t = w_i^{t-1} - \eta \frac{\boxed{r_t}}{\boxed{\sqrt{v_t} + \epsilon}}$$

Momentum

Escalamiento del momentum

Alternativas a SGD



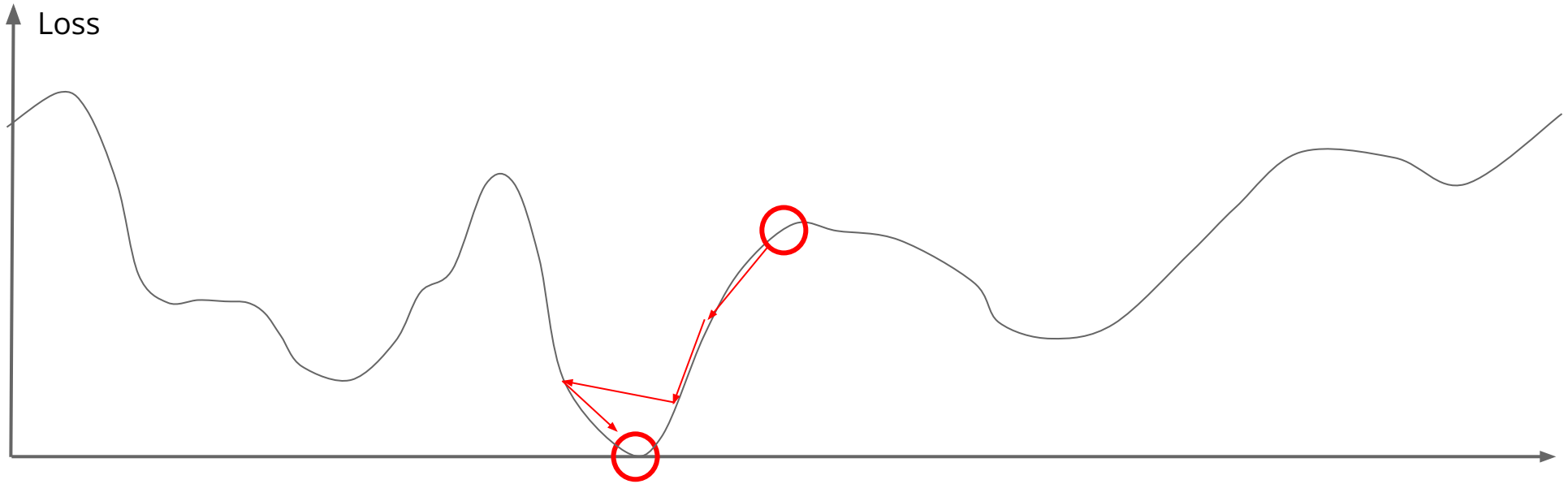
Adaptar el Learning Rate

- ¿Cómo podemos definir el learning rate que necesitamos?
  - Quizás necesitamos diferentes valores en diferentes etapas del entrenamiento.

- Decaimiento del learning rate.
  - En base a la cantidad de épocas.
  - En base al valor de la función de pérdida del set de validación.

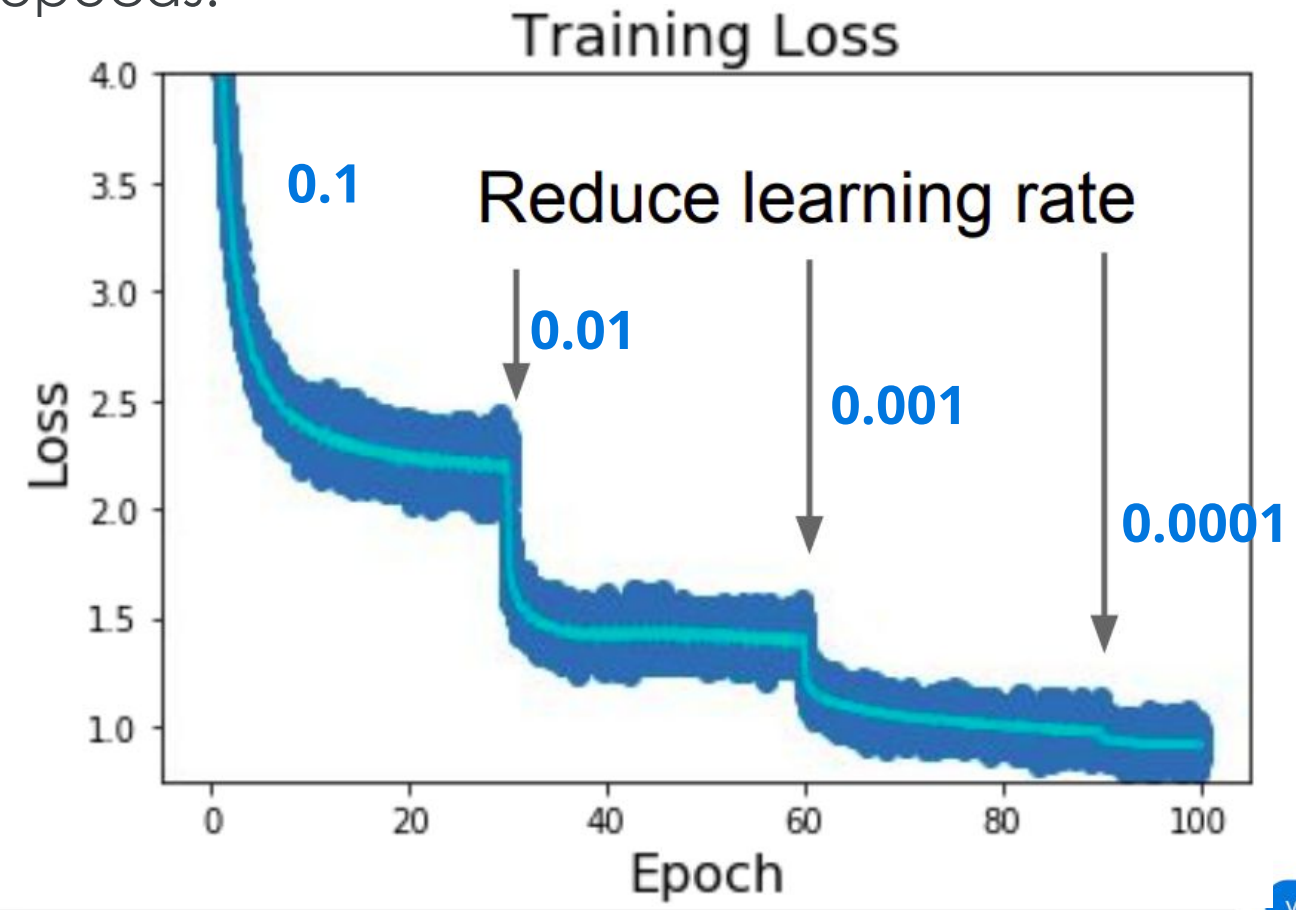
Adaptar el Learning Rate

- ¿Cómo podemos definir el learning rate que necesitamos?
  - Quizás necesitamos diferentes valores en diferentes etapas del entrenamiento.



Adaptar el Learning Rate

- Entrenamiento de una ResNet, multiplicando por 0.1 el Learning Rate cada 30 épocas.



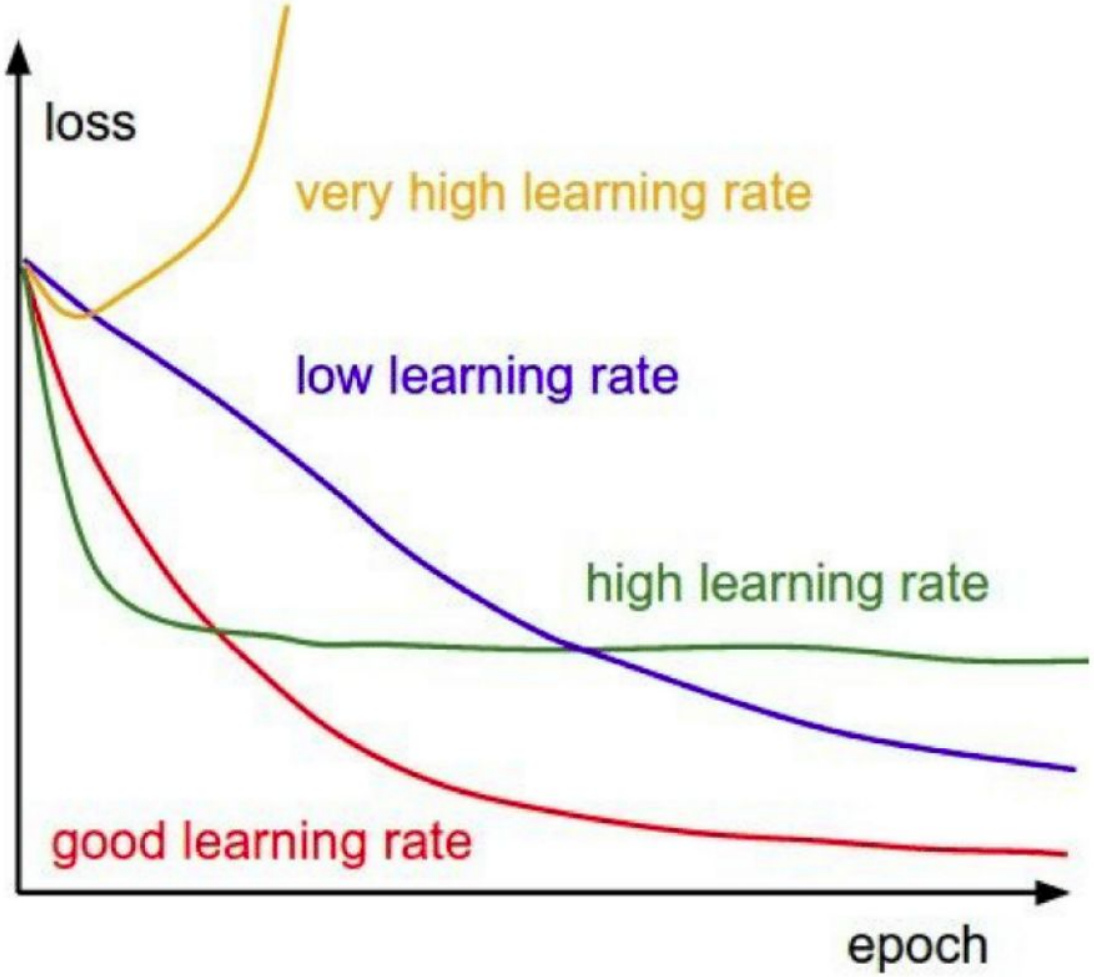
Adaptar el Learning Rate

- En base al número de épocas.
  - Cada cierta cantidad de épocas, reducir el valor del learning rate por un factor.

- Utilizando el conjunto de validación.
  - Verificar si el valor de la función de pérdida no baja en n épocas, reducir el valor del learning rate.

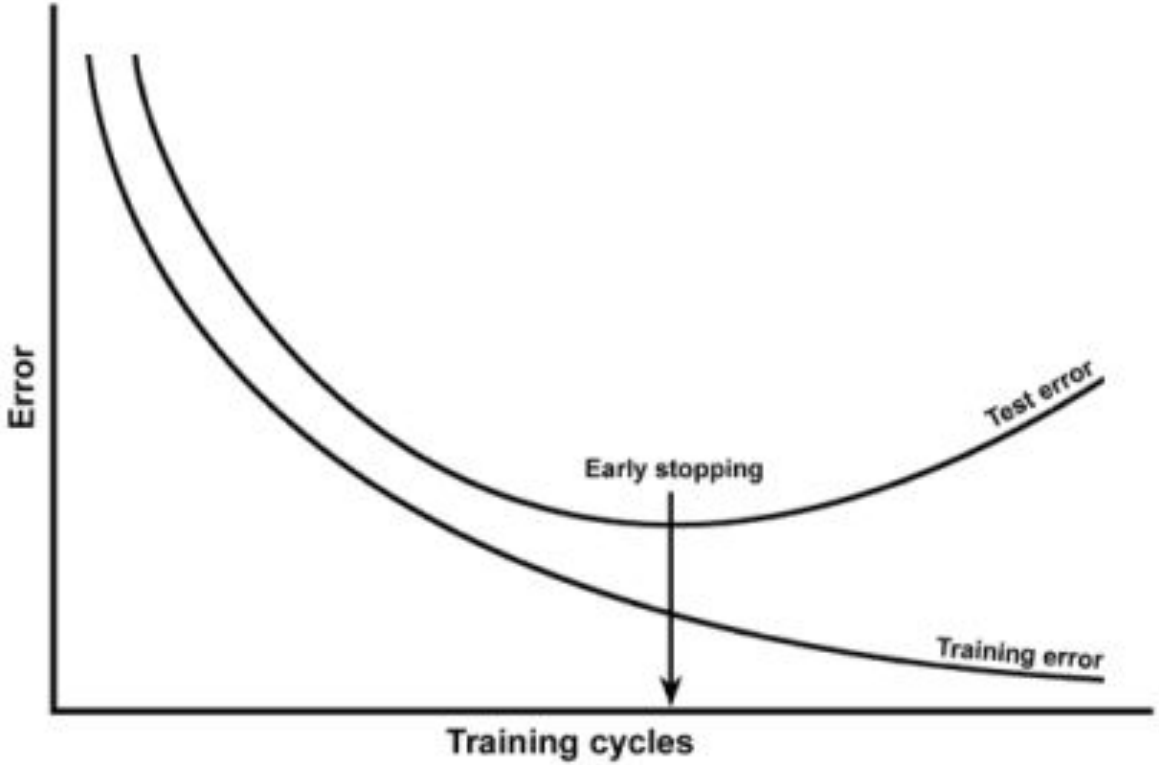
- Otras opciones que pueden depender del problema.

Adaptar el Learning Rate



Adaptar el Learning Rate

- Early stopping.
  - Si durante el entrenamiento vemos que el conjunto de validación está empeorando, podemos detener el entrenamiento.



Preguntas

Extras: Nuevos optimizadores

- **2015 - ADAM**
  - https://arxiv.org/pdf/1412.6980v9.pdf
- **2018 - ADMM and Accelerated ADMM as Continuous Dynamical Systems**
  - https://arxiv.org/pdf/1805.06579.pdf
- **2019 - Ranger**
  - https://github.com/lessw2020/Ranger-Deep-Learning-Optimizer
- **2019 - Lookahead**
  - https://arxiv.org/pdf/1907.08610v1.pdf
- **2020 - Gradient Centralization**
  - https://arxiv.org/pdf/2004.01461v2.pdf
- **2121 - AngularGrad**
  - https://arxiv.org/pdf/2105.10190.pdf

ESCUELA DE INGENIERÍA
FACULTAD DE INGENIERÍA

EDUCACIÓN PROFESIONAL

www.educacionprofesional.ing.uc.cl